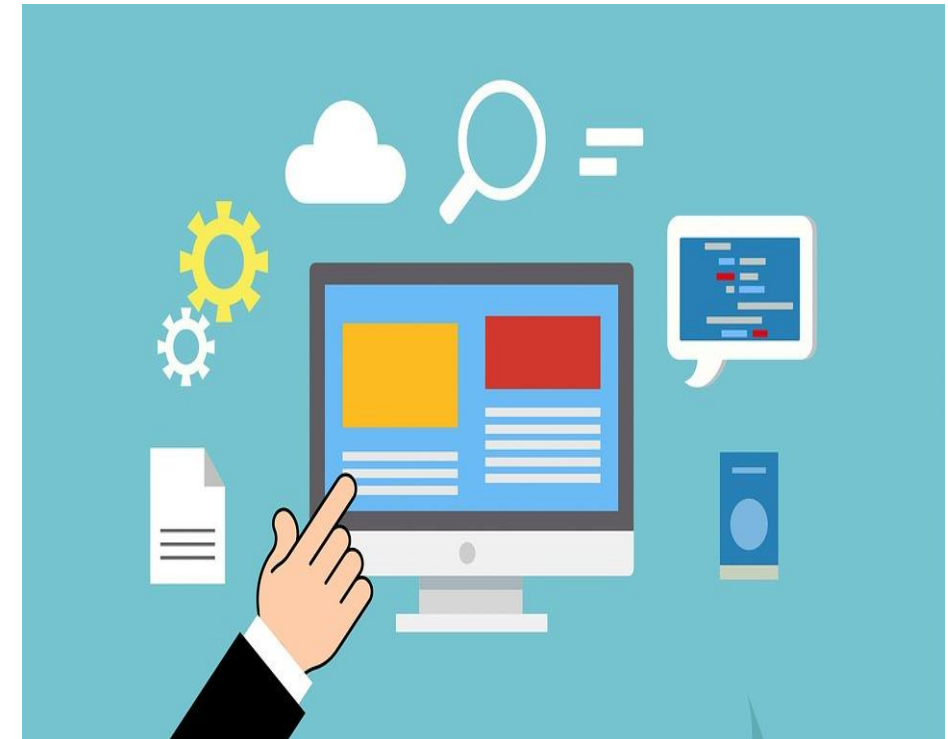


DESARROLLO DE SISTEMAS WEB

Licenciatura en Ingeniería de Ciberseguridad e
Infraestructura de Cómputo

```
else if (i==2)
{
    var atpos=inputs[i].indexof("@");
    var dotpos=inputs[i].lastindexOf(".");
    if (atpos<1 || dotpos<atpos+1 || dotpos==inputs[i].length-1)
        document.getElementById("error").innerHTML+=
            "El correo electrónico no es válido. " +
            "El correo electrónico debe tener un símbolo @ y un dominio " +
            "separado por un punto. Ejemplo: usuario@dominio.com" +
            "<br>";
    else
        document.getElementById("div").innerHTML+=
            "El correo electrónico es válido. " +
            "Ejemplo: usuario@dominio.com" +
            "<br>";
}
```



Fuentes de información

Unidad 3. Estándares para el desarrollo web

- Shklar Leon y Rosen Richard. Web Application, Principles, Protocols and Practices. Wiley. 2009.
- Gasston, Peter. The Modern Web: Multi-Device Web Development with HTML5, CSS3, and JavaScript. No Starch Press. 2013.
- Jennifer Niederst Robbins. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics. O'REILLY. 2012.

Saberes teóricos

Unidad 3. Estándares para el desarrollo web

III. Estándares para el desarrollo web

- Accesibilidad de contenidos y posicionamiento en la web
- Normas de accesibilidad de aplicaciones web
- HTML semántico y posicionamiento en buscadores
- Evolución y sintaxis de HTML
- Hojas de estilo en cascada
- Lenguajes de scripting
- El modelo de DOM
- Manipulación de documentos
- Javascript Object Notation
- Javascript Asíncrono y XML (Ajax)

¿Qué es JavaScript?

Lenguajes de scripting

- JavaScript es un robusto lenguaje de programación que puede ser aplicado a un documento HTML y usado para crear interactividad dinámica en los sitios web.
- **Su nombre oficial es ECMAScript.**
- Su primera versión fue lanzada en 1995, junto con Netscape Navigator 2.0.
- En 1997 se envió la especificación JavaScript 1.1 al organismo ECMA (European Computer Manufacturers Association).
- JavaScript es la implementación que realizó la empresa Netscape del estándar ECMAScript ([ECMA-262](#)).

Características

Lenguajes de scripting

- JavaScript es una de las tecnologías de **front-end** mas importantes.
- Le permite a una pagina contar con características dinámicas.
- JavaScript cuenta con los mecanismos para navegar y modificar la estructura de un documento HTML.
- JavaScript esta dirigido por eventos: el usuario al interactuar con la pagina, genera constantemente eventos que JavaScript puede procesar.

Características

Lenguajes de scripting

- Es un lenguaje interpretado (el navegador trae por defecto un interprete JavaScript).
- Es un lenguaje multiparadigma (estructurado, funcional, parcialmente orientado a objetos).
- Es de tipificación dinámica.
- Es mayormente imperativo (instrucción paso a paso).
- Cuenta con una sintaxis similar a C y no tiene nada que ver con Java de Oracle.
- Esta basado en el estándar ECMAScript.

Características

Lenguajes de scripting

JavaScript es el lenguaje de programación más popular del mundo.

JavaScript es el lenguaje de programación de la Web.

JavaScript es fácil de aprender.



Lenguaje de Scripting

Sintaxis de JavaScript

- Ejemplo. Cuando el usuario hace clic en el botón, se ejecuta el script.

```
<button onclick="saludar()">Saludar</button>
```

```
<script>  
function saludar() {  
    alert("Bienvenido al portal");  
}  
</script>
```

JavaScript permite:

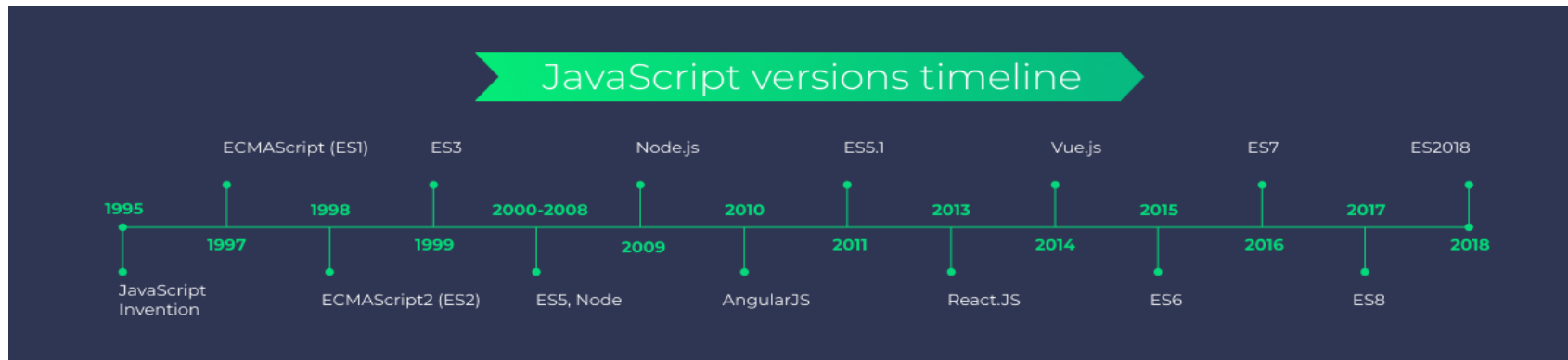
- Interactuar con el usuario
- Modificar el DOM
- Comunicarse con servidores
- Responder a eventos

Evolución

Lenguajes de scripting

- Su primera versión fue lanzada en 1995, junto con Netscape Navigator 2.0.
- Se convirtió en un estándar ECMA en 1997.
- Las versiones de ECMAScript se han abreviado como ES1, ES2, ES3, ES5 y ES6.
 - JavaScript original ES1 ES2 ES3 (1997-1999).
 - Primera revisión principal ES5 (2009).
 - Segunda Revisión ES6 (2015).
 - Última revisión ES14 (2023).
- Desde 2016, las versiones se nombran por año (ECMAScript 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023).

ECMAScript 1 (1997)
ECMAScript 2 (1998)
ECMAScript 3 (1999)
ECMAScript 4 (Never Released)
ECMAScript 5 (2009)
ECMAScript 5.1 (2011)
ECMAScript 6 (2015)
ECMAScript 2016 (ES7)
ECMAScript 2017 (ES8)
ECMAScript 2018 (ES9)
ECMAScript 2019 (ES10)
ECMAScript 2020 (ES11)
ECMAScript 2021 (ES12)
ECMAScript 2022 (ES13)
ECMAScript 2023 (ES14)



¿Cómo se agrega a una página Web?

Lenguajes de scripting

- Se puede agregar de tres formas:
- 1. **Incrustado**. Los scripts se pueden colocar en la sección <body>, o en el elemento <head> de una página HTML, o en ambas, mediante la etiqueta <script>.
- 2. **Inline**. Se puede agregar de forma inline como parte del manejo de un atributo de evento:

```
<button onclick="mifuncion('hola');">Enviar</ button>
```

- 3. **Archivo externo**. De preferencia utilizar un archivo de script externo.

```
<script src="miScript.js"></script>
```

```
<script src="otroScript.js"></script>
```

Incrustado

Lenguajes de scripting

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    alert("Hola mundo!");
  </script>
</head>
<body>
</body>
</html>
```

EN LA CABECERA (ETIQUETA <HEAD>)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    alert("Hola mundo!");
  </script>
</body>
</html>
```

EN EL BODY (ETIQUETA <BODY>)

Inline

Lenguajes de scripting

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <button type="button" onclick="alert('Hola mundo!');">Dame clic</button>
</body>
</html>
```

Archivo externo

Lenguajes de scripting

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="holamundo.js"></script>
</head>
<body>
</body>
</html>
```

EN LA CABECERA (ETIQUETA <HEAD>)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script src="holamundo.js"></script>
</body>
</html>
```

EN EL BODY (ETIQUETA <BODY>)

Ventajas de JavaScript externo

Lenguajes de scripting

- Los scripts externos son prácticos cuando se utiliza el mismo código en muchas páginas web diferentes.
- Los archivos JavaScript tienen la extensión de archivo .js.
- Separa HTML y código.
- Hace que HTML y JavaScript sean más fáciles de leer y mantener.
- Los archivos JavaScript almacenados en caché pueden acelerar la carga de la página.
- Reduce el tamaño del archivo HTML.
- Funciona bien con sistemas modernos de control de versiones de código fuente como GIT. Lo que significa que cada uno de estos archivos mantendrá un historial y varios programadores podrán registrarlo y retirarlo.
- Muchos paquetes de JavaScript populares están disponibles alojados en redes de entrega de contenido (CDN) y puede simplemente señalarlos usando la URL en el src, evitando así copiar el archivo js a la carpeta local.

Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

Objetivo: Comprender que JavaScript se ejecuta en el navegador y permite añadir comportamiento a una página web.

1. Crea una carpeta destinada para "Ejercicios javascript".
2. Descarga la página index.html. (De la actividad en Eminus "Demostración javascript")
3. Crea una subcarpeta css y guarda en esta el archivo estilos.css (Descárgalo de la actividad en Eminus "Demostración javascript").

Debe quedar una página como la siguiente:

Tienda de Videojuegos

[Inicio](#) [Catálogo](#) [Ofertas](#) [Contacto](#)

Catálogo

Legend of Adventure

Precio: \$60

Comprar

Seleccione un videojuego para comprar.

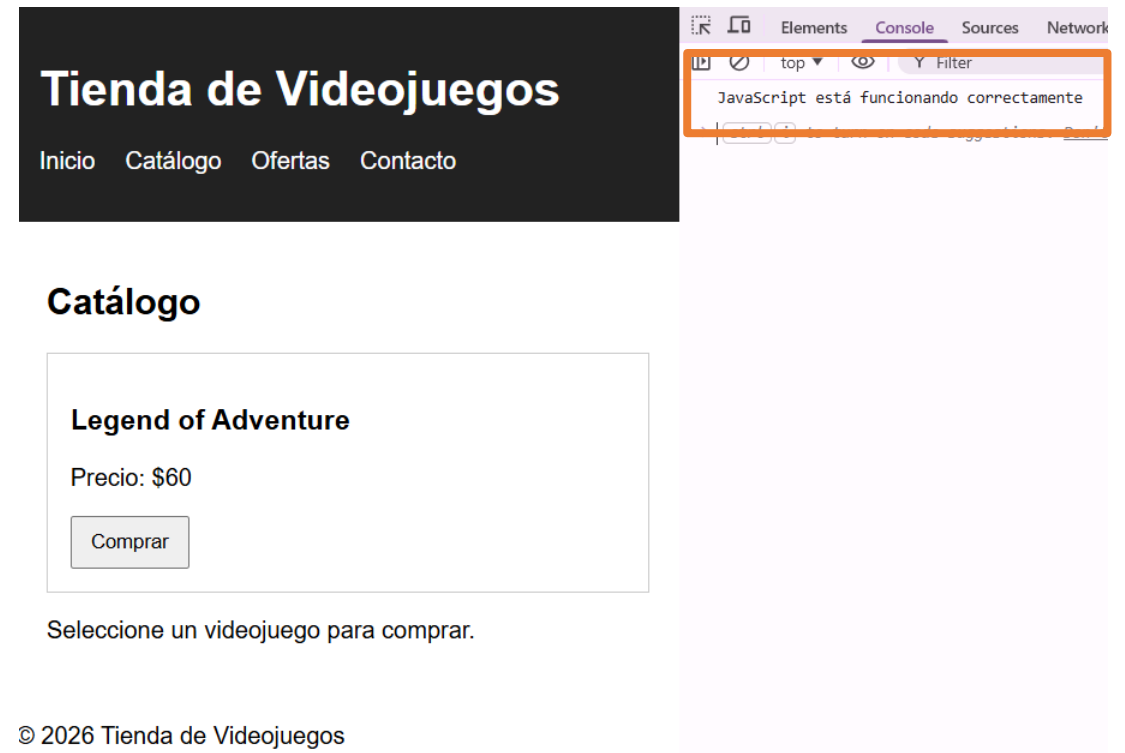
Demostración

Lenguajes de scripting

- Crea una subcarpeta llamada js y dentro, un archivo llamado *script.js*.
- Dentro de este archivo copia las líneas de código siguientes

```
console.log("JavaScript está funcionando correctamente");
```

Abre la página en el navegador y en la consola aparecerá:



Orden de ejecución de JavaScript

Lenguajes de scripting

- Cuando el navegador encuentra un bloque de JavaScript, generalmente lo ejecuta en orden, de arriba a abajo.
- Esto significa que **debes tener cuidado con el orden** en el que se colocan las instrucciones.
- Es un lenguaje de programación **interpretado** ligero. El navegador web recibe el código JavaScript en su forma de texto original y ejecuta el script a partir de ahí.
- Técnicamente, la mayoría de los intérpretes de JavaScript modernos utilizan una técnica llamada compilación en tiempo real para mejorar el rendimiento: el código fuente de JavaScript se compila en un formato binario más rápido mientras se usa el script, de modo que se pueda ejecutar lo más rápido posible.
- Sin embargo, JavaScript todavía se considera un lenguaje interpretado, ya que la compilación se maneja en el entorno de ejecución, en lugar de antes.

Estrategias para la carga de scripts

Lenguajes de scripting

- Para evitar que una instrucción se ejecute antes de que se cargue el cuerpo HTML, se puede usar un detector/manejador de eventos.

```
document.addEventListener("DOMContentLoaded", function() {  
    ...  
});
```

- El evento "DOMContentLoaded" del navegador significa que el cuerpo HTML está completamente cargado y analizado. El JavaScript dentro de este bloque no se ejecutará hasta que se active ese evento.

Estrategias para la carga de scripts

Lenguajes de scripting

- Una solución antigua a este problema solía ser colocar el elemento script **justo en la parte inferior del cuerpo** (por ejemplo, justo antes de la etiqueta `</body>`), para que se cargara después de haber procesado todo el HTML.
- El problema con esta solución es que la carga/procesamiento del script está completamente bloqueado hasta que se haya cargado el DOM HTML.
- En sitios muy grandes con mucho JavaScript, esto puede causar un importante problema de rendimiento y ralentizar el sitio.
- El atributo **defer** en una etiqueta `<script>` de HTML es una técnica de optimización que le dice al navegador: *"Descarga este script en segundo plano mientras continúas cargando el HTML, pero no lo ejecutes hasta que todo el documento HTML se haya analizado por completo"*

async

Estrategias para la carga de scripts

- **async.** Los scripts cargados con el atributo async descargarán el script sin bloquear el renderizado de la página y lo ejecutará **tan pronto** como el script se termine de descargar.
- No se tiene garantía de que los scripts se ejecuten en un orden específico, solo que no detendrán la visualización del resto de la página.
- Es mejor usar async cuando los scripts de la página se ejecutan de forma independiente y no dependen de ningún otro script de la página.

```
<script async src="js/vendor/jquery.js"></script>  
  
<script async src="js/script2.js"></script>  
  
<script async src="js/script3.js"></script>
```

- **No se puede confiar en el orden** en que se cargarán los scripts. jquery.js se puede cargar antes o después de script2.js y script3.js y si este es el caso, cualquier función en esos scripts dependiendo de jquery producirá un error, porque jquery no se definirá en el momento en que se ejecute el script.

defer

Estrategias para la carga de scripts

- **defer**. Se debe usar cuando se tiene una gran cantidad de scripts en segundo plano para cargar, se desea ponerlos en su lugar lo antes posible.
- No bloquea la carga de la página HTML pues al cargar el script se descargan en un hilo separado al resto de la página HTML.
- Los scripts cargados con el atributo defer se ejecutarán tan pronto como se descarguen **pero hasta que este todo** el contenido HTML.
- Además se ejecutarán **en el orden** en que aparecen en la página.

```
<script defer src="js/vendor/jquery.js"></script>
```

```
<script defer src="js/script2.js"></script>
```

```
<script defer src="js/script3.js"></script>
```

defer

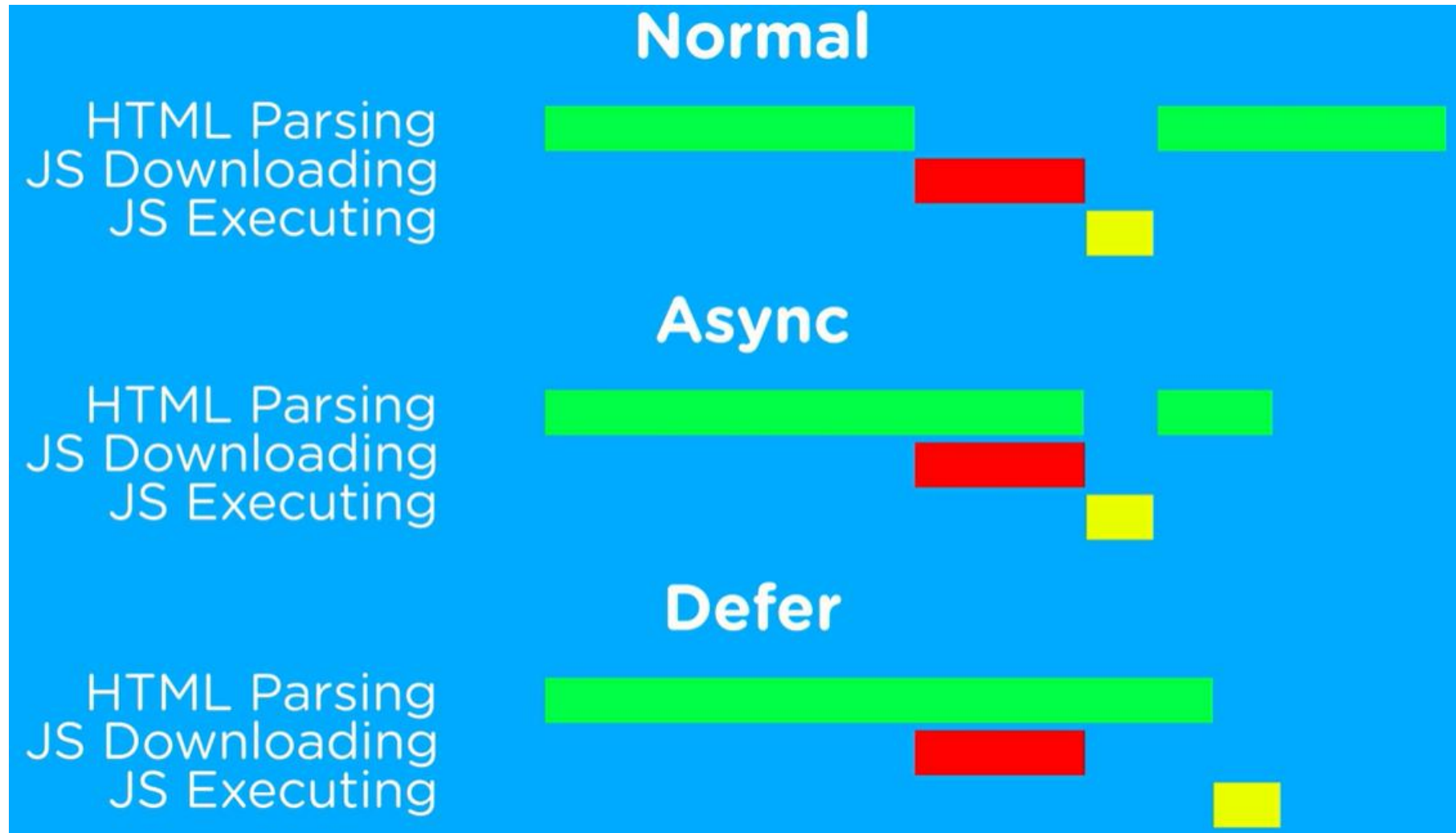
Estrategias para la carga de scripts

Por lo tanto, si no se emplea **defer**:

- **Bloqueo del renderizado (Render-blocking):** El analizador de HTML (HTML parser) se detiene al encontrar la etiqueta `<script>` y no continúa leyendo el resto del documento hasta que el script se descarga y ejecuta.
- **Carga lenta:** La página web puede mostrarse en blanco o tardar más en mostrar contenido al usuario, lo que afecta la experiencia de usuario (UX) y el rendimiento.
- **Errores de manipulación del DOM:** Si el JavaScript intenta modificar elementos de la página (`document.getElementById`, etc.) y el script se carga antes de que el HTML de esos elementos haya sido procesado, el script fallará.
- **Orden de ejecución:** Los scripts sin `defer` o `async` se ejecutan en el orden en que aparecen en el código, pero pausando la carga del sitio por cada uno

async y defer

Estrategias para la carga de scripts



Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

Objetivo: Mostrar dónde se integra el atributo **defer**.

- Observa la etiqueta `<script>` dentro del `<head>`:

```
<script src="js/script.js"></script>
```

De esta forma, el navegador detiene la carga del HTML para descargar y ejecutar el script inmediatamente. Esto bloquea la renderización, provocando una carga más lenta de la página y posibles errores si el script intenta acceder a elementos HTML que aún no existen

Demostración

Lenguajes de scripting

Agregamos:

```
<script src="js/script.js" defer></script>
```

De esta forma el navegador:

- Descarga el script
- Termina de cargar el HTML
- Ejecuta el script

Sintaxis de JavaScript

Lenguajes de scripting

Sintaxis de JavaScript

Sintaxis de JavaScript

- La sintaxis de JavaScript es el conjunto de reglas que dictan cómo se construyen los programas de JavaScript:

```
let x, y, z;      // Como se declaran las variables  
x = 5; y = 6;     // Como asignar valores  
z = x + y;        // Como calcular valores
```

- Cuenta con una sintaxis similar a C.
- Ya que es un lenguaje muy extenso, la mejor manera de aprender la sintaxis es de **manera práctica** con ejemplos.
- <https://developer.mozilla.org/en-US/docs/Learn/JavaScript>
- https://www.w3schools.com/js/js_intro.asp

Variables

Sintaxis de JavaScript

- La sintaxis de JavaScript define dos tipos de valores: valores fijos y valores variables.
 - Los valores fijos se llaman **constantes**.
 - Los valores variables se llaman **variables**.

Variables

Sintaxis de JavaScript

- Las variables son contenedores para almacenar datos.
- Las variables de JavaScript se pueden declarar de 3 formas:
 - Usando **var**.

```
var x = 5;  
var y = 6;  
var z = x + y;
```

```
const price1 = 5;  
const price2 = 6;  
let total = price1 + price2;
```

- Usando **let**.
- Usando **const**.
- La palabra clave **var** se utilizó en todo el código JavaScript desde 1995. hasta 2015.
- Las palabras clave **let** y **const** se agregaron a JavaScript en 2015.
- Las sentencias terminan con ;
- Los valores literales y operadores son similares a Java o C.

Variables

Sintaxis de JavaScript

- En general, debe limitarse a usar caracteres latinos (0-9, a-z, A-Z) y el carácter de guion bajo.
- No usar guiones bajos al comienzo de los nombres de las variables.
- No usar números al comienzo de las variables. Esto no está permitido y provoca un error.
- Una convención segura a seguir es la llamada "minúscula mayúsculas intercaladas", en la que se juntan varias palabras con minúsculas para la primera palabra completa y luego en mayúsculas las primeras letras de las siguientes palabras.
- Haz que los nombres de las variables sean intuitivos, para que describan los datos que contienen. No uses solo letras/números o frases grandes y largas.
- Las variables distinguen entre mayúsculas y minúsculas.
- Evitar el uso de palabras reservadas de JavaScript como nombres de variables.

Tipos de datos

Sintaxis de JavaScript

- **Número.** Valores numéricos enteros y decimales.
- **String.** Cadenas de caracteres.
- **Boolean:** true o false
- **Object.** Declaraciones de objetos.
- **Array.** Arreglos de valores.
- **Date.** Fechas.
- **Null:** nulo
- **Undefined:** no definido.
- Es un lenguaje de **tipado dinámico**, lo cual significa que, a diferencia de otros lenguajes, no es necesario especificar qué tipo de datos que contendrá una variable.

```
let myAge = 17;
```

```
let dolphinGoodbye = "Hasta luego y gracias por todos los peces";
```

```
let iAmAlive = true;
```

```
let dog = { name: "Spot", breed: "Dalmatian" };
```

```
let myNameArray = ["Chris", "Bob", "Jim"];
```

```
let myNumberArray = [10, 15, 40];
```

```
// foo no existe, no está definido y nunca ha sido inicializado:
```

```
> foo
```

```
"ReferenceError: foo is not defined"
```

```
// foo existe, pero no tiene tipo ni valor:
```

```
> var foo = null; foo
```

```
"null"
```


Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

Objetivo: Declarar variables en JavaScript.

Agregar en script.js:

```
let nombreJuego = "Legend of Adventure";  
let precio = 60;  
let disponible = true;
```

```
console.log(nombreJuego);  
console.log(precio);  
console.log(disponible);
```

Demostración

Lenguajes de scripting

Aparecerán los valores asignados en consola:

Tienda de Videojuegos

[Inicio](#) [Catálogo](#) [Ofertas](#) [Contacto](#)

Catálogo

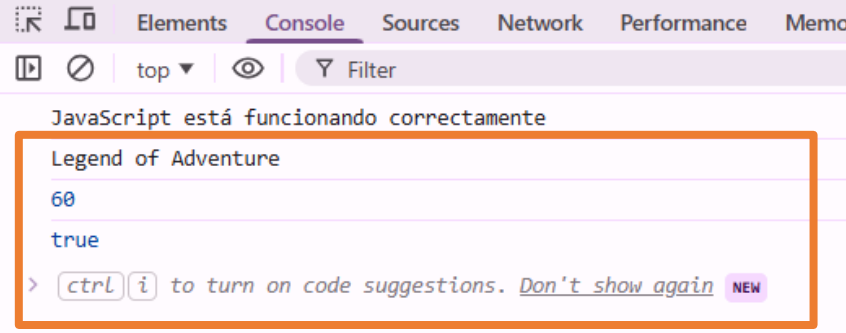
Legend of Adventure

Precio: \$60

Comprar

Seleccione un videojuego para comprar.

© 2026 Tienda de Videojuegos



```
JavaScript está funcionando correctamente
Legend of Adventure
60
true
> ctrl i to turn on code suggestions. Don't show again NEW
```

Operadores de comparación

Sintaxis de JavaScript

Operador	Nombre	Propósito	Ejemplo
<code>===</code>	Igual estricto	Comprueba si los valores izquierdo y derecho son idénticos entre sí	<code>5 === 2 + 4</code>
<code>!==</code>	Igual no-estricto	Comprueba si los valores izquierdo y derecho no son idénticos entre sí	<code>5 !== 2 + 3</code>
<code><</code>	Menor que	Comprueba si el valor izquierdo es menor que el derecho.	<code>10 < 6</code>
<code>></code>	Mayor que	Comprueba si el valor izquierdo es mayor que el derecho.	<code>10 > 20</code>
<code><=</code>	Menor o igual a	Comprueba si el valor izquierdo es menor o igual que el derecho.	<code>3 <= 2</code>
<code>>=</code>	Mayor o igual a	Comprueba si el valor izquierdo es mayor o igual que el derecho..	<code>5 >= 4</code>

Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

Objetivo: Comparar valores para tomar decisiones.

Agregar en script.js:

```
let precioJuego = 60;
```

```
if(precioJuego > 50){  
  console.log("Este videojuego es premium");  
}
```

Demostración

Lenguajes de scripting

En consola aparecerá:

- Este videojuego es premium

The image shows a web application interface on the left and a browser's developer console on the right. The web application has a dark header with the title 'Tienda de Videojuegos' and navigation links: 'Inicio', 'Catálogo', 'Ofertas', and 'Contacto'. Below the header is a 'Catálogo' section featuring a card for 'Legend of Adventure' with a price of '\$60' and a 'Comprar' button. At the bottom of the catalog, it says 'Seleccione un videojuego para comprar.' and '© 2026 Tienda de Videojuegos'. The browser console on the right shows several messages: 'JavaScript está funcionando correctamente', 'Legend of Adventure', '60', and 'true'. A message 'Este videojuego es premium' is highlighted with an orange box. Below it, a system message suggests using 'ctrl i' for code suggestions.

Tienda de Videojuegos

Inicio Catálogo Ofertas Contacto

Catálogo

Legend of Adventure

Precio: \$60

Comprar

Seleccione un videojuego para comprar.

© 2026 Tienda de Videojuegos

JavaScript está funcionando correctamente

Legend of Adventure

60

true

Este videojuego es premium

> `ctrl i` to turn on code suggestions. [Don't show again](#) NEW

Cadenas

Sintaxis de JavaScript

Es una variable que tiene cero o más caracteres escritos entre comillas.

- Pueden ser comillas simples ''.
- Pueden ser comillas dobles "".
- Pueden ser comillas invertidas ``. (A partir del 2015).

```
let nombre = 'visitante';  
console.log('Hola ' + nombre + ', bienvenido al sitio.');
```



```
console.log('Hola "vistante", bienvenido al sitio.');
```



```
console.log("Hola " + nombre + ", bienvenido al sitio.");
```



```
console.log(`Hola ${nombre}, bienvenido al sitio.`);
```

https://developer.mozilla.org/es/docs/Learn/JavaScript/First_steps/Useful_string_methods

Demostración

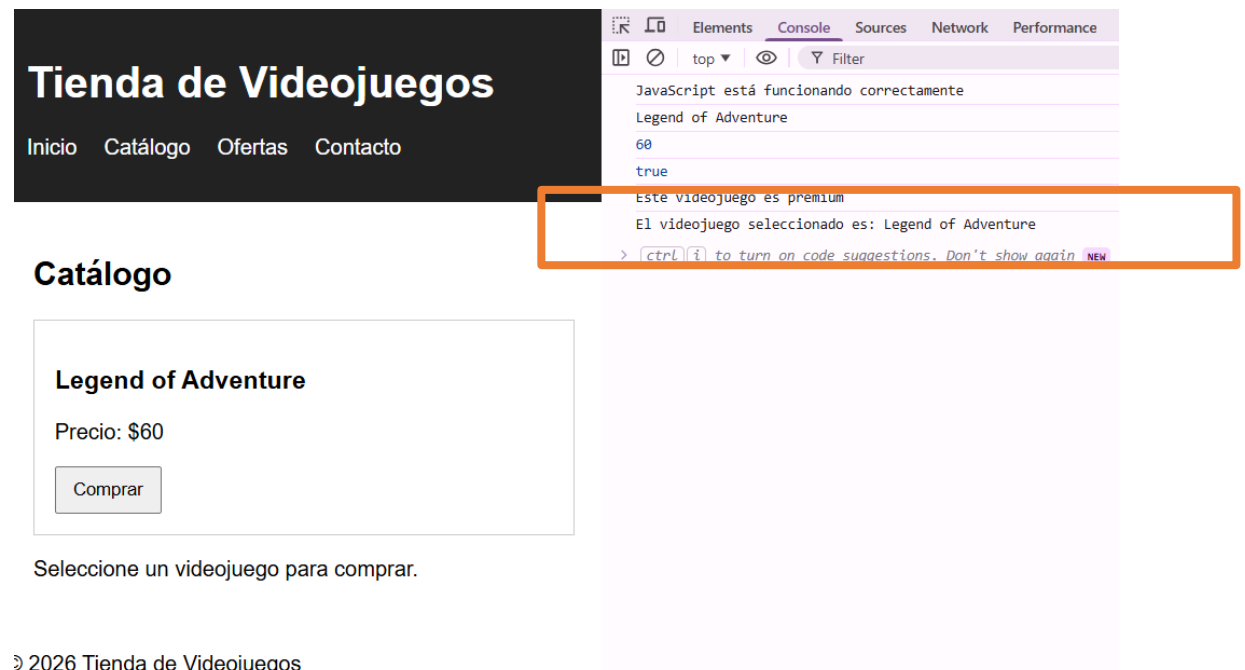
Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

Objetivo: Manipular texto en JavaScript.

Agregar en script.js:

```
let juego = "Legend of Adventure";  
let mensaje = "El videojuego seleccionado es: " + juego;  
console.log(mensaje);
```



Demostración

Lenguajes de scripting

También se puede mostrar con template strings:

Agregar:

```
let mensaje2 = `El videojuego seleccionado es: ${juego}`;  
console.log(mensaje2);
```

Tienda de Videojuegos

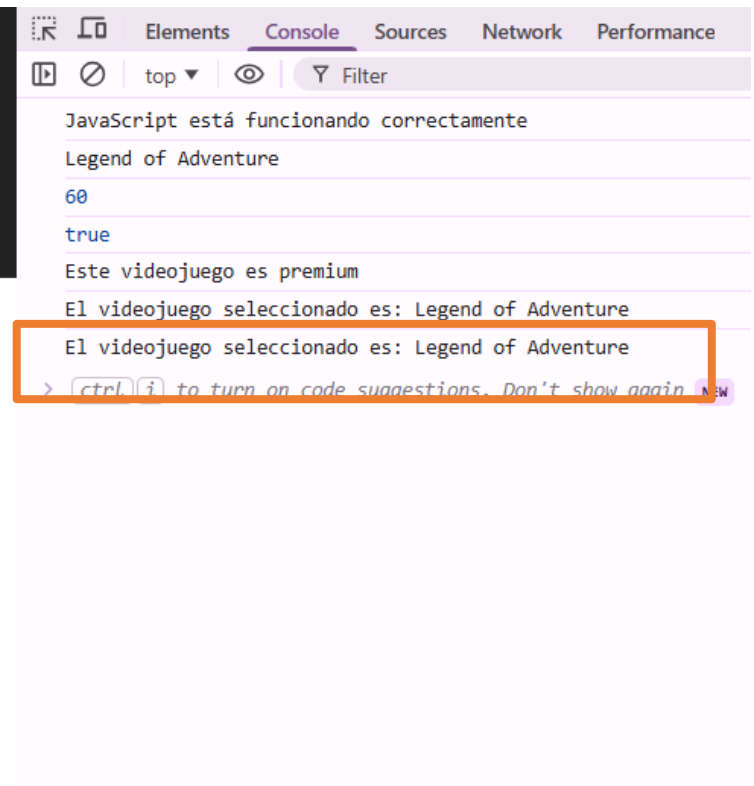
[Inicio](#) [Catálogo](#) [Ofertas](#) [Contacto](#)

Catálogo

Legend of Adventure

Precio: \$60

Comprar



Funciones

Sintaxis de JavaScript

- Hay varias formas de crear funciones en JavaScript:
 - Por declaración.
 - Por expresión.
 - Mediante constructor de objeto.

Constructor	Descripción
FUNCTION <code>function nombre(p1, p2...) { }</code>	Crea una función mediante declaración .
FUNCTION <code>var nombre = function(p1, p2...) { }</code>	Crea una función mediante expresión .
FUNCTION <code>new Function(p1, p2..., code);</code>	Crea una función mediante un constructor de objeto .

Funciones por declaración

Funciones

- Es la forma más popular de las tres. Esta forma permite declarar una función que existirá a lo largo de todo el código:

```
function saludar() {  
    return "Hola";  
}
```

```
saludar(); // 'Hola'  
typeof saludar; // 'function'
```

Funciones por expresión

Funciones

- En JavaScript es muy habitual encontrarse códigos donde los programadores «guardan funciones» dentro de variables, para posteriormente «ejecutar dichas variables». Se trata de un enfoque diferente, creación de funciones por expresión.

```
// El segundo "saludar" (nombre de la función) se suele omitir: es redundante
const saludo = function saludar() {
    return "Hola";
};

saludo(); // 'Hola'
```

- El nombre de la función (en este ejemplo: saludar) pasa a ser inútil, ya que si intentamos ejecutar saludar() nos dirá que no existe y si intentamos ejecutar saludo() funciona correctamente.
- ¿Qué ha pasado? Ahora el nombre de la función pasa a ser el nombre de la variable, mientras que el nombre de la función desaparece y se omite, dando paso a lo que se llaman las **funciones anónimas (o funciones lambda)**.

Funciones por expresión vs declaración

Funciones

- La función por expresión se ejecuta escribiendo la variable con paréntesis.
- La diferencia fundamental entre las funciones por declaración y las funciones por expresión es que estas últimas sólo están disponibles a partir de la inicialización de la variable.
- Si «ejecutamos la variable» antes de declararla, nos dará un error.

```
// saludo no se puede llamar antes
const saludo = function saludar() {
    return "Hola";
};

saludo(); // 'Hola'
```

```
// Se puede llamar antes
saludar(); // 'Hola'

function saludar() {
    return "Hola";
}
```

Funciones autoejecutables

Funciones

- Pueden existir casos en los que necesites crear una función y ejecutarla sobre la marcha. En JavaScript es muy sencillo crear funciones autoejecutables. Sólo se debe envolver entre paréntesis la función anónima en cuestión (no es necesario que tenga nombre, puesto que no la vamos a guardar) y luego, ejecutarla:

```
// Función autoejecutable
(function () {
    console.log("Hola!!");
})();
```

```
// Función autoejecutable con parámetros
(function (name) {
    console.log(`¡Hola, ${name}!`);
})("Fei");
```

Funciones Arrow (A partir de 2015)

Funciones

- Las Arrow functions, funciones flecha o «fat arrow» son una forma corta de escribir funciones que aparece en Javascript a partir de ECMAScript 6.
- Básicamente, se trata de reemplazar eliminar la palabra function y añadir => antes de abrir las llaves:

```
const func = function () {  
    return "Función tradicional.";   
};
```

```
const func = () => {  
    return "Función flecha.";   
};
```

```
const func = () => "Función flecha."; // 0 parámetros: Devuelve "Función flecha"
```

```
const func = (e) => e + 1; // 1 parámetro: Devuelve el valor de e + 1
```

```
const func = (a, b) => a + b; // 2 parámetros: Devuelve el valor de a + b
```

Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

Objetivo: Crear funciones para reutilizar código.

Agregar en script.js:

```
function comprarJuego() {  
    document.getElementById("info").textContent =  
        "Has agregado el juego al carrito";  
}
```

Luego añadir el evento al botón:

```
document.getElementById("btnComprar")  
    .addEventListener("click", comprarJuego);
```

Demostración

Lenguajes de scripting

El mensaje de texto en la página cambiará:

Tienda de Videojuegos

[Inicio](#) [Catálogo](#) [Ofertas](#) [Contacto](#)

Catálogo

Legend of Adventure

Precio: \$60

Comprar

Has agregado el juego al carrito

© 2026 Tienda de Videojuegos

Extra: Elimina el atributo **defer** de la etiqueta `<script>`, recarga la página y observa la consola.

Módulos

JavaScript

- JavaScript no tenía ninguna forma de cargar código desde otros archivos JavaScript. Todo el JavaScript se tenía que escribir en un sólo archivo .js.
- A partir de **ECMAScript** (2015) se introduce una característica nativa denominada Módulos ES (ESM), que permite la importación y exportación de fragmentos de datos entre diferentes ficheros JavaScript, eliminando las desventajas que se tenían y permitiendo trabajar de forma más flexible.

Palabras clave

Módulos

- Para trabajar con módulos tenemos a nuestra disposición las siguientes **palabras clave**:

Declaración	Descripción
export	Pone los datos indicados (variables, funciones, clases...) a disposición de otros ficheros.
import	Incorpora datos (variables, funciones, clases...) desde otros ficheros .js al código actual.
import()	Permite importar módulos de forma más flexible, en tiempo real (imports dinámicos).

- Para poder utilizar **export** o **import** en JavaScript, se debe cargar el fichero .js con la etiqueta y atributo **<script type="module">** para indicarle que se utilizarán módulos.

export

Módulos

- Mediante la palabra clave **export** crearemos lo que se llama un módulo de exportación que contiene datos. Estos datos pueden ser variables, funciones, clases u objetos más complejos.

Declaración	Descripción
export ...	Declara un elemento o dato, a la vez que lo añade al módulo de exportación.
export {name}	Añade el elemento name al módulo de exportación.
export {name as newName}	Añade el elemento name al módulo de exportación con el nombre newName.
export { n1, n2, n3... }	Añade los elementos indicados (n1 , n2 , n3 ...) al módulo de exportación.
export * from "./file.js"	Añade todos los elementos del módulo de file.js al módulo de exportación.
export default ...	Declara un elemento y lo añade como módulo de exportación por defecto .

export

Módulos

Declaración y exportación

```
export let number = 42;           // Se añade la variable number al módulo
export const hello = () => "Hello!"; // Se añade la función hello al módulo
export class CodeBlock { };      // Se añade la clase vacía CodeBlock al módulo
```

Exportación post-declaración

```
let number = 42;
const hello = () => "Hello!";
const goodbye = () => "¡Adiós!";
class CodeBlock { };

export { number };           // Se crea un módulo y se añade number
export { hello, goodbye as bye }; // Se añade saludar y despedir al módulo
export { hello as greet };   // Se añade otroNombre al módulo
```

```
let number = 42;
const hello = () => "Hello!";
const goodbye = () => "¡Adiós!";
class CodeBlock { };

export {
  number,
  hello,
  goodbye as bye,
  hello as greet
};
```

import

Módulos

- Con la palabra clave **import** podremos leer dichos módulos exportados desde otros ficheros y utilizar sus elementos en el código de nuestro fichero actual.

Declaración	Descripción
<code>import { nombre } from "./file.js"</code>	Importa el elemento nombre de file.js.
<code>import { nombre as newName } from "./file.js"</code>	Importa el elemento nombre de file.js como newName.
<code>import { n1, n2... } from "./file.js"</code>	Importa los elementos indicados desde file.js.
<code>import nombre from "./file.js"</code>	Importa el elemento por defecto de file.js como nombre.
<code>import * as name from "./file.js"</code>	Importa todos los elementos de file.js en el objeto name.
<code>import "./file.js"</code>	Ejecuta el código de file.js. No importa ningún elemento.
<code>import { name } from "https://web.com/file.js"</code>	Descarga el fichero e importa el elemento name de su módulo.

import

Módulos

Importación nombrada.

```
import { nombre } from "./file.js";  
import { number, element } from "./file.js";  
import { brand as brandName } from "./file.js";
```

Importación por defecto. Una importación por defecto lo único que hace es buscar el elemento llamado default e importarlo con el nombre indicado en el **import**.

```
import nombre from "./math.js";
```

Observe como aquí no se utilizan las llaves.

import

Módulos

Importación de código. Indica al navegador debe leer el código de ese fichero y procesarlo, sin importar ningún elemento como en los casos anteriores.

```
import "./math.js";
```

Importaciones remotas.

```
import { ceil } from "https://unpkg.com/lodash-es@4.17.21/lodash.js";
```

import

Módulos

Importación dinámica. A diferencia del **import** estático, este fichero no se cargará siempre y desde el principio, sino que sólo lo hará cuando se llegue a esta parte del código, siendo posible incluirla dentro de condicionales, funciones o lógica diversa.

```
import("./module.js")           // Dynamic import
.then(data => { /* ... */ });
```

```
// Opción 1: Se carga functions.js si se cumple la condición
if (number > 42) {
  import("./functions.js")
    .then(module => module.func());
}

// Opción 2: Se carga functions.js interpolando la constante
const filename = "functions";
import(`./${filename}.js`)
  .then(module => module.func());

// Opción 3: Se carga additional.js sólo si el usuario hace click en el botón
const button = document.querySelector("button.info");
button.addEventListener("click", () => import("additional.js"), { once: true });
```


Demostración

Lenguajes de scripting

Objetivo: Comprender cómo dividir el código en módulos.

Crea el archivo js/utilidades.js y copia el siguiente código en él:

```
export function calcularPrecioConImpuesto(precio) {  
    return precio * 1.16;  
}
```

Modifica en script.js:

```
import { calcularPrecioConImpuesto } from "./utilidades.js";  
let precioFinal = calcularPrecioConImpuesto(60);  
console.log(precioFinal);
```

- En consola aparecerá:

69.6

Arreglos

Sintaxis de JavaScript

- Es una variable especial que puede contener más de un valor.
- Son en realidad objetos.

```
// Usando corchetes
```

```
const cars = ["Audi", "Volvo", "BMW"];  
const audi = cars [0];
```

```
// Usando la palabra clave new
```

```
const cars = new Array ("Audi", "Volvo", "BMW");  
const longitud = cars.length; // Obtiene la cantidad de elementos
```

```
// Puede quedar vacío
```

```
const cars = [];  
const cars= new Array();
```

```
// Se puede crear como un diccionario "clave->valor"
```

```
const diccionario = [];  
diccionario ["nombre"] = "pepe";
```

Arreglos

Sintaxis de JavaScript

• Operaciones

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
console.log(fruits);
```

```
console.log(fruits.toString()); // Banana,Orange,Apple,Mango
```

```
console.log(fruits.join("-")); // Banana-Orange-Apple-Mango
```

```
fruits.pop(); // remueve ultimo  
console.log(fruits);
```

```
fruits.push("Kiwi"); // agrega al final  
console.log(fruits);
```

```
fruits.shift(); // remueve primero  
console.log(fruits);
```

```
fruits.unshift("Lemon"); // agrega al inicio  
console.log(fruits);
```

```
delete fruits[0]; // cambia a undefined el primero  
console.log(fruits);
```

► (4) ['Banana', 'Orange', 'Apple', 'Mango']

Banana,Orange,Apple,Mango

Banana-Orange-Apple-Mango

► (3) ['Banana', 'Orange', 'Apple']

► (4) ['Banana', 'Orange', 'Apple', 'Kiwi']

► (3) ['Orange', 'Apple', 'Kiwi']

► (4) ['Lemon', 'Orange', 'Apple', 'Kiwi']

► (4) [vacío, 'Orange', 'Apple', 'Kiwi']

Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

- **Objetivo:** Trabajar con colecciones de datos.
- Agregar en script.js:

```
let juegos = ["Zelda", "Mario Kart", "FIFA", "Halo"];
```

```
console.log(juegos);  
console.log(juegos[0]);
```

Agregar un nuevo juego:

```
juegos.push("Minecraft");  
console.log(juegos);
```

Sentencias de control

Sintaxis de JavaScript

```
const time = new Date().getHours();
let saludo = "";
if (time < 12) {
    saludo = "Buen día";
} else if (time < 20) {
    saludo = "Buena tarde";
} else {
    saludo = "Buena noche";
}
console.log(saludo);
```

```
var text = "";
switch (new Date().getDay()) {
    case 6:
        text = "Hoy es sábado";
        break;
    case 0:
        text = "Hoy es domingo";
        break;
    default:
        text = "Estoy esperando el fin de semana";
}
console.log(text);
```

Sentencias de control

Sintaxis de JavaScript

```
let i = 0;
while (i < 5) {
    console.log("El número es " + i);
    i++;
}
```

```
i = 0;
do {
    console.log("El número es " + i);
    i++;
}
while (i < 5);
```

```
i = 0;
for (i = 0; i < 5; i++) {
    console.log("El número es " + i);
}
```

El número es 0

El número es 1

El número es 2

El número es 3

El número es 4

El número es 0

El número es 1

El número es 2

El número es 3

El número es 4

El número es 0

El número es 1

El número es 2

El número es 3

El número es 4

Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

- **Objetivo:** Tomar decisiones en el código.
- Agregar en script.js:

```
precio=40;  
if(precio < 50){  
  console.log("Oferta disponible");  
}else{  
  console.log("Precio normal");  
}
```

Demostración

Lenguajes de scripting

```
let categoria = "accion";

switch(categoria){
case "accion":
console.log("Juegos de acción");
break;

case "deportes":
console.log("Juegos deportivos");
break;

default:
console.log("Categoría no encontrada");
}
```


Objetos

Sintaxis de JavaScript

- Un objeto es una colección de datos relacionados y/o funcionalidad (que generalmente consta de algunas variables y funciones, que se denominan propiedades y métodos cuando están dentro de objetos).

```
const person = { firstName: "John", lastName: "Doe", age: 50, eyeColor: "blue" };
```

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

```
console.log(person.lastName);  
console.log(person["lastName"]);
```

Objetos

Sintaxis de JavaScript

- Se pueden declarar funciones dentro de objetos.

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  id: 5566,  
  fullName: function () {  
    return this.firstName + " " + this.lastName;  
  }  
};  
  
let name = person.fullName();
```

- No declarar cadenas, números y valores booleanos como objetos.

```
let x = new String();  
let y = new Number();  
let z = new Boolean();
```

- Complican el código y ralentizan la velocidad de ejecución.

Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

- **Objetivo:** Representar entidades del mundo real.

- Agregar en script.js:

```
juego = {  
  nombre: "Zelda",  
  precio: 60,  
  disponible: true  
};
```

```
console.log(juego.nombre);  
console.log(juego.precio);
```

```
document.getElementById("info").  
textContent =  
`Juego: ${juego.nombre} -  
Precio: ${juego.precio}`;
```

Clases

Sintaxis de JavaScript

- ECMAScript 2015, también conocido como ES6, introdujo clases de JavaScript.
- Las clases de JavaScript son plantillas para objetos de JavaScript.

```
class ClassName {  
    constructor() { ... }  
    method_1() { ... }  
    method_2() { ... }  
    method_3() { ... }  
}
```

Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

- **Objetivo:** Crear estructuras reutilizables.
- Agregar en script.js:

```
class Videojuego {  
    constructor(nombre, precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
    mostrarInfo() {  
        return `${this.nombre} cuesta ${this.precio}`;  
    }  
}
```

Crear instancia:

```
let juego1 = new Videojuego("Halo",  
70);  
  
console.log(juego1.mostrarInfo());
```

Forms

Manipulación documentos

- El elemento HTML form (<form>) representa una sección de un documento que contiene controles interactivos que permiten a un usuario enviar información a un servidor web.

```
<form>
  <div class="mb-3">
    <label for="exampleInputEmail1" class="form-label">Email address</label>
    <input type="email" class="form-control" id="exampleInputEmail1">
    <div id="emailHelp" class="form-text">Escriba su correo UV.</div>
  </div>
  <div class="mb-3">
    <label for="exampleInputPassword1" class="form-label">Password</label>
    <input type="password" class="form-control" id="exampleInputPassword1">
  </div>
  <div class="mb-3 form-check">
    <input type="checkbox" class="form-check-input" id="exampleCheck1">
    <label class="form-check-label" for="exampleCheck1">Acepto las condiciones</label>
  </div>
  <button type="submit" class="btn btn-primary">Entrar</button>
</form>
```

Email address

Escriba su correo UV.

Password

☐ Acepto las condiciones

Entrar

Atributos

Forms

Atributo	Descripción
accept-charset	Especifica las codificaciones de caracteres utilizadas para el envío de formularios.
action	Especifica dónde enviar los datos del formulario cuando se envía un formulario.
autocomplete	Indica cuales de los controles en este formulario puede tener sus valores automáticamente completados por el navegador. Puede estar en off o en on.
enctype	Especifica cómo se deben codificar los datos del formulario al enviarlos al servidor (solo para method="POST")
method	Especifica el método HTTP que se utilizará al enviar datos del formulario. GET o POST.
name	Especifica el nombre del formulario.
novalidate	Este atributo booleano indica que el formulario no es validado cuando es enviado. Si el atributo no existe formnovalidate en un <button> o en un elemento <input> que pertenece al formulario
rel	Especifica la relación entre un recurso vinculado y el documento actual.
target	Especifica dónde mostrar la respuesta que se recibe después de enviar el formulario.

```
<form name="miForma" action="validaentrada.html" method="post" target="_self"
autocomplete="off" novalidate accept-charset="UTF-8" enctype="multipart/form-data">
```

Elementos

Forms

- El elemento **<datalist>**. Representa la lista de elementos `<option>` como sugerencias cuando se llena un campo `<input>`
- El elemento **<input>**. Es un elemento de texto básico; tiene varios valores para el atributo `type` (text, password, search, tel, url, email, range, color).
- El elemento **<output>** representa el resultado de un cálculo.
- **<label>** representa una etiqueta para un elemento en una interfaz de usuario. Este puede estar asociado con un control ya sea mediante la utilización del atributo `for`, o ubicando el control dentro del elemento `label`.
- **<textarea>** representa un campo para la entrada de texto multilínea. El control asociado a este campo es una caja de texto que permite a los usuarios editar múltiples líneas de texto regular.

Elementos

Forms

- **<button>**. Un botón puede ser de tres tipos: submit, reset, o button.
 - Un clic en un botón **submit** envía la información del formulario a una pagina web definida por defecto en el atributo **action** del elemento **<form>**.
 - Un clic en un botón **reset** reinicia inmediatamente todos los widgets del formulario a sus valores por defecto. Desde un punto de viste de UX, esto se considera una mala práctica.

Demostración

Lenguajes de scripting

Objetivo: Crear Forms con elementos funcionales

Paso 1 – Agregar el siguiente formulario en HTML, antes de cerrar `</main>`

```
<section>
```

```
<h2>Registro de compra</h2>
```

```
<form id="formCompra">
```

```
<label>Nombre:</label>
```

```
<input type="text" id="nombreUsuario" required>
```

```
<label>Juego:</label>
```

```
<input type="text" id="juegoSeleccionado" required>
```

```
<button type="submit">Enviar</button>
```

```
</form>
```

```
</section>
```

Demostración

Lenguajes de scripting

Paso 2- Capturar datos con JavaScript

//Código para formulario

```
document.getElementById("formCompra") //Selecciona el elemento formulario formCompra
    .addEventListener("submit", function (e) { //Añade un escuchador de eventos para el envío
        e.preventDefault(); //Evita que la página se recargue
        let nombre = document.getElementById("nombreUsuario").value; //Recibe los datos del
formulario en variables
        let juego = document.getElementById("juegoSeleccionado").value;
        document.getElementById("info").textContent =
            `Compra realizada por ${nombre} - Juego: ${juego}`;
```

Demostración

Lenguajes de scripting

Tienda de Videojuegos

[Inicio](#)[Catálogo](#)[Ofertas](#)[Contacto](#)

Catálogo

Legend of Adventure

Precio: \$60

Comprar

Compra realizada por Erika - Juego: Mario Bros

Registro de compra

Nombre:

Juego:

Enviar

Try/catch

Sintaxis de JavaScript

- Manejo de errores.

```
let x = "";
try {
  if (x.trim() == "") throw "esta vacío";
  if (isNaN(x)) throw "no es un número";
  x = Number(x);
  if (x > 10) throw "es muy alto";
  if (x < 5) throw "es muy bajo";
}
catch (err) {
  console.log("Error: " + err + ".");
}
finally {
  console.log("Se ejecuta sin importar si hubo error o no.");
}
```

```
try {
  Block of code to try
}
catch(err) {
  Block of code to handle errors
}
finally {
  Block of code to be executed regardless of the try / catch result
}
```

Error: esta vacío.

Se ejecuta sin importar si hubo error o no.

Demostración

Lenguajes de scripting

Nombre: Ejercicios para demostración del tema Lenguaje de scripting

- **Objetivo:** Manejar errores en JavaScript.
- Agregar en script.js:

```
let datos = "texto incorrecto";
```

```
JSON.parse(datos); //Convierte una cadena con  
formato JSON "datos" en un objeto utilizable de  
JavaScript. Una cadena con formato json es  
similar a: {"juego": "Zelda"}
```

Demostración

Lenguajes de scripting

Si manejamos el error con try/catch

```
try {  
  let datos = '{"juego": "Zelda"}';  
  
  let objeto = JSON.parse(datos);  
  
  document.getElementById("info").textContent =  
    `Juego: ${objeto.juego}`;  
  
} catch (error) {  
  
  document.getElementById("info").textContent =  
    "Error al cargar el juego";  
  
}
```

Si cambiamos nuevamente la línea:

```
let datos = "texto incorrecto";
```

¿Qué sucedió con el error?

Comentarios

Sintaxis de JavaScript

- Hay dos tipos:

- Un comentario de una sola línea se escribe después de una doble barra inclinada (//), p. ej.

```
JS // soy un comentario
```

- Se escribe un comentario de varias líneas entre las cadenas /* y */, p. ej.

```
JS /*  
  Yo también soy  
  un comentario  
*/
```




Preguntas